International Institute of Information Technology
Bangalore (IIITB)

Domestic Flight Price Prediction

Param Pandey (MT2026065)
Nabodaya Mete (MT2026058)
Prakhar Srivastava (MT2026071)
Varun Gupta (MT2026031)

Machine Learning Project Report

15 October 2026

Contents

Executive Summary **iii**

1 Introduction **1**

2 Dataset Overview **1**
2.1 Target Variable 1
2.2 Input Features 2

3 Preprocessing Pipeline **2**
3.1 Handling Missing Values 2
3.2 Encoding of Categorical Features 3
3.3 Feature Scaling 3
3.4 Construction of a ColumnTransformer 4

4 Exploratory Data Analysis **4**

5 Feature Engineering **7**
5.1 Log-Transformed Features 7
5.2 Ratio Features 7
5.3 Interaction Features 7

6 Train–Validation Split **8**

7 Supervised Models (Full Dataset) **8**
7.1 Models and settings 9
7.2 Round-1 data (`flight_ML`) 9
7.3 Round-2 data (`flight_ML2`) 10
7.4 Reading the results 10

8 Final Model Selection **12**
8.1 Why this blend 12
8.2 Hyper-parameter tuning (`flight_ml3`) 12
8.3 Error analysis of the final blend 13

9 Threshold Optimization **15**
9.1 Blend weight 15
9.2 Tuned members in the blend 15
9.3 Rare-code cut-off 16

10 Kaggle Submission **16**
10.1 Predictions on the test set 16

11 Experiments with 20% Dataset **17**

12 Unsupervised Learning **18**

13 Key Insights & Discussion **19**

14 Limitations **19**

15 Future Work **20**

16 Conclusion **20**

References **21**

Appendix A. Kaggle Public Leaderboard History **21**

Appendix B. Artifact Traceability **22**

List of Figures

1 Price before and after the log1p transform 4
2 Price by class, stops, departure time and airline code 5
3 Days left and duration against price 5
4 Correlation matrix of numeric features 6
5 Train vs test distributions 6
6 Missing values in train and test 6
7 Round-1 cross-validation comparison 9
8 Round 1 vs round 2 cross-validation 11
9 Error analysis of the final blend 13
10 Test-set predictions by days left and feature importance 14
11 Blend weight sweep 15
12 Kaggle public leaderboard scores 16
13 Predicted test prices vs training prices 17
14 Learning curve of the blend 17
15 PCA and k-means 18

List of Tables

1 Raw input features (training set) 2
2 Duration-imputation models (round 2) 3
3 Encoding of features (round 1) 3
4 Effect of seven extra features on the XGB + ET blend 7
5 Train and hold-out splits for both rounds 8
6 Models compared and their settings 9
7 Round 1: 20% hold-out results 9
8 Round 1: 5-fold cross-validation 10
9 Round 2: 20% hold-out results 10
10 Round 2: 5-fold cross-validation 11
11 Hyper-parameter tuning results 12
12 Hold-out comparison after the widened search 13
13 Out-of-fold error by group 13
14 Blend weight results 15
15 Predicted test medians by class 16
16 Models trained on 20% of the training split 17
17 The two k-means clusters 18
18 Kaggle public leaderboard history 21

Executive Summary

Our best Kaggle submission (public score **0.96238**) is an equal-weight blend of XGBoost and Extra Trees, trained on the first-round preprocessed data and predicting log-price. The second preprocessing round (ML-imputed duration, rare-airline grouping) scored slightly lower (0.96077), and hyper-parameter tuning gave mixed results: Extra Trees improved, XGBoost did not.

Report at a Glance	
Task	Supervised regression: predict flight ticket price (Rs)
Training data	15,589 rows × 12 columns (11 features + price)
Test data	3,898 rows × 11 columns (no price)
Model input (round 1)	14,431 de-duplicated rows × 65 engineered features
Target transform	`log1p(price)`; predictions mapped back with `expm1`
Validation	80/20 split stratified by class + 5-fold cross-validation
Models compared	9 single models + 3 ensembles, on two preprocessing rounds
Final model	`VotingRegressor(XGBoost, Extra Trees)`, round-1 data
CV result (final model)	log RMSE 0.1352, R^2 0.9817, MAE Rs 3,339, MAPE 7.9%
Best public Kaggle score	0.96238

Scope statement: All Kaggle scores and the model and tuning tables in Sections 7–8 are copied from the three project notebooks (`flight_ML`, `flight_ML2`, `flight_ml3`). Sections 5, 9, 11 and 12 add analyses run afterwards in the consolidated notebook; they are labelled *[retrospective]* and did not influence the submitted model. All figures except Figure 12 were generated by the consolidated notebook *Flight_Price_Prediction_Final.ipynb* from `train.csv` and `test.csv`. Hidden-test scores exist only for the submissions listed in Appendix A.

1 Introduction

The goal is to predict the ticket price of a domestic flight from its airline, flight code, route, timing, number of stops, cabin class, duration and days left until departure. Accurate price prediction helps travellers decide when to book and helps sellers understand which factors drive fares.

The work ran in three stages, each in its own notebook:

1. `flight_ML` — compares baseline, linear, tree and boosting models on the first-round preprocessed data. The best model is an equal-weight blend of XGBoost and Extra Trees.
2. `flight_ML2` — repeats the same comparison on the second-round preprocessed data. Its best blend scored below the round-1 blend on Kaggle, so round 1 was kept.
3. `flight_ml3` — tunes the hyper-parameters of the two blend members with randomized search.

All three notebooks, together with both preprocessing notebooks, were merged into one consolidated notebook (*Flight_Price_Prediction_Final.ipynb*). That notebook is the basis for this report; it also regenerates every figure and the final test-set predictions.

2 Dataset Overview

The training file has 15,589 flights with a known price; the test file has 3,898 flights whose prices are scored on Kaggle. City and airline names are anonymised (for example *Metropolis*, *Wayfarer Aviation*).

2.1 Target Variable

`price` is the fare in rupees. It ranges from Rs 3,801 to Rs 323,177, with a median of Rs 10,183 and a mean of Rs 29,817. The distribution is strongly right-skewed and bimodal because of cabin class: Economy fares average Rs 8,818, Business fares Rs 60,127. All models therefore predict `log1p(price)`, which makes the target near-symmetric and turns errors into relative errors.

2.2 Input Features

Feature	Type	Distinct	Missing (train / test)	Notes
`id`	integer	15,589	0 / 0	Row identifier; correlation with price 0.002
`airline`	categorical	18	3,089 / 799	Two airlines hold 66% of rows
`flight`	string	1,070	0 / 0	2-letter prefix + number, e.g. *IX 1070*; 19 prefixes
`source_city`	categorical	6	0 / 0	Metropolis is the busiest origin (6,198)
`destination_city`	categorical	6	0 / 0	Night City is the busiest destination (4,663)
`departure_time`	categorical	6	0 / 0	Early_Morning … Late_Night
`arrival_time`	categorical	6	0 / 0	Same six time bands
`stops`	ordinal	3	0 / 0	zero 10,965 · one 4,558 · two_or_more 66
`flight_class`	binary	2	0 / 0	Economy 9,209 · Business 6,380
`duration`	numeric (h)	—	3,095 / 776	0.92–38.08 h, median 2.5 h
`days_left`	numeric (days)	57	0 / 0	1–57 days before departure

Table 1: Raw input features of the training set, with type, number of distinct values, missing counts and notes.

3 Preprocessing Pipeline

Two preprocessing rounds were built. Round 1 (14,431 rows × 65 features) produced the best Kaggle score and is used by the final model; round 2 (14,697 rows × 58 features) tested a model-based duration fill and grouping of rare airlines. Every look-up table and imputation model is learned on the training file only and then applied unchanged to the test file.

3.1 Handling Missing Values

Airline (3,089 train / 799 test missing). The first two characters of `flight` identify the airline with 100% purity: no prefix maps to more than one airline. All 3,089 missing training values were filled from the prefix, which also showed that `airline` adds nothing beyond the prefix, so it was dropped.

Duration (3,095 train / 776 test missing). Duration matters (correlation with price 0.699), so rows were not dropped.

- *Round 1 — hierarchical median.* Fill with the median duration of the same flight + route + stops; fall back to route + stops, then route, then the global median. On a held-out 20% of known durations the first level covered 95.76% of rows with MAE 0.188 h (RMSE 1.383 h).

- *Round 2 — predictive model.* Six regressors predicted duration from flight code, cities, times, stops, class and flight number. Extra Trees had the lowest 5-fold out-of-fold MAE and filled the gaps.

Model	**MAE (h)**
Extra Trees	0.109
Random Forest	0.125
XGBoost	0.134
LightGBM	0.148
Hist Gradient Boosting	0.224
CatBoost	0.229

Table 2: Duration-imputation models in round 2, ranked by 5-fold out-of-fold MAE.

Other columns. `id` was dropped (correlation with price 0.002). Dropping `flight_number` created exact duplicates, which were removed from the training data (1,158 rows in round 1, 892 in round 2).

3.2 Encoding of Categorical Features

Feature	**Encoding**	**Columns (round 1)**
`stops`	Ordinal: zero / one / two_or_more → 0 / 1 / 2	1
`flight_class`	Binary: Economy → 0, Business → 1	1
`departure_time`, `arrival_time`	One-hot	6 + 6
`flight` prefix (airline code)	One-hot	19
`source_city` × `destination_city`	One-hot of the directional route (A→B differs from B→A)	30
`duration`, `days_left`	Numeric, unchanged	2

Table 3: Encoding applied to each feature in round 1 and the number of model columns it produces.

In round 2, airline codes with fewer than 50 rows (9I, RA, S5, KU, SQ, FZ, WY, GF — 92 rows in total) were merged into `flight_code_other`, leaving 12 code columns. The test set contains one code never seen in training (BG, 1 row); 29 test rows fall into rare or unseen codes.

3.3 Feature Scaling

Tree-based models are insensitive to monotone rescaling, so features were left unscaled for them. Ridge regression penalises coefficient size, so it runs inside `make_pipeline(StandardScaler(), Ridge(alpha=1.0))`; the scaler is fitted on the training fold only, which avoids leakage:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}.$$

3.4 Construction of a ColumnTransformer

The project notebooks encoded with `pd.get_dummies` and aligned test columns by re-indexing. For the consolidated notebook the same steps were also written as one scikit-learn `ColumnTransformer` *[retrospective]*:

- `OneHotEncoder(handle_unknown="ignore")` on `departure_time`, `arrival_time`, prefix and route — unseen test codes such as BG become all-zero rows instead of errors;
- `OrdinalEncoder` with fixed category order on `stops` and `flight_class`;
- `StandardScaler` on `duration` and `days_left`.

It yields the same 65 features and can be placed in a `Pipeline` in front of any model, so preprocessing is re-fitted inside every cross-validation fold.

Cross-validated on all 14,431 rows, the ColumnTransformer versions score log RMSE 0.2500 (Ridge) and 0.1326 (XGB + ET blend), in line with the `get_dummies` versions. These runs use more rows per fold than Table 8, so the numbers are not directly comparable.

4 Exploratory Data Analysis

Cabin class dominates price: it alone explains about 76% of the variance of log-price (the class-mean baseline in Section 7 reaches R^2 0.764). Within each class, stops, duration, airline code and departure slot explain most of what remains, while `days_left` has a small, non-linear effect.

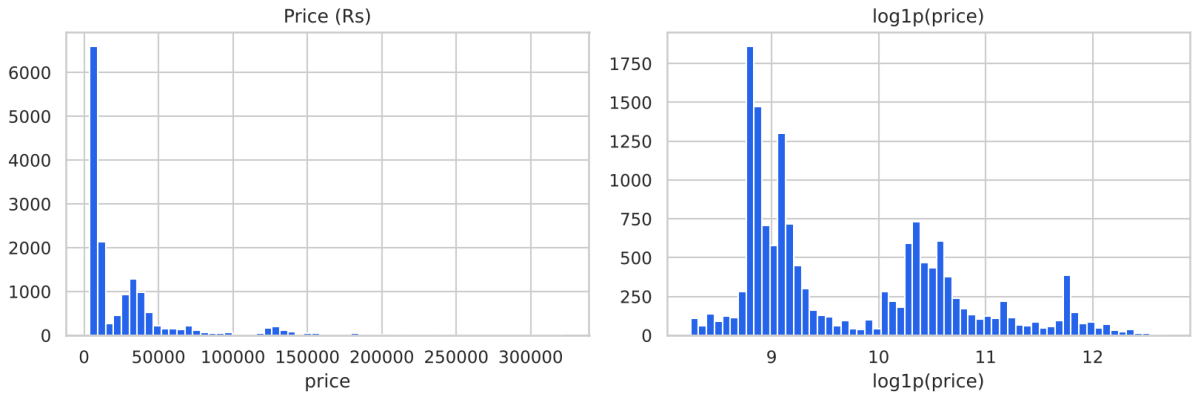


Figure 1: Raw price is right-skewed and bimodal; `log1p(price)` is close to two symmetric humps, one per class.

- **Class.** Median Business fare is about 5× the Economy median (Rs 38,546 vs Rs 7,587).
- **Stops.** Non-stop flights have a median of Rs 8,883, one-stop Rs 40,528, two-or-more Rs 125,611. Much of this is a class effect: 77% of one-stop flights are Business, and 64 of the 66 two-or-more-stop flights are Business.
- **Departure time.** Late-night departures have a median of Rs 33,067 against Rs 9,450–11,000 for every other slot; again this reflects which airlines and classes fly at night.

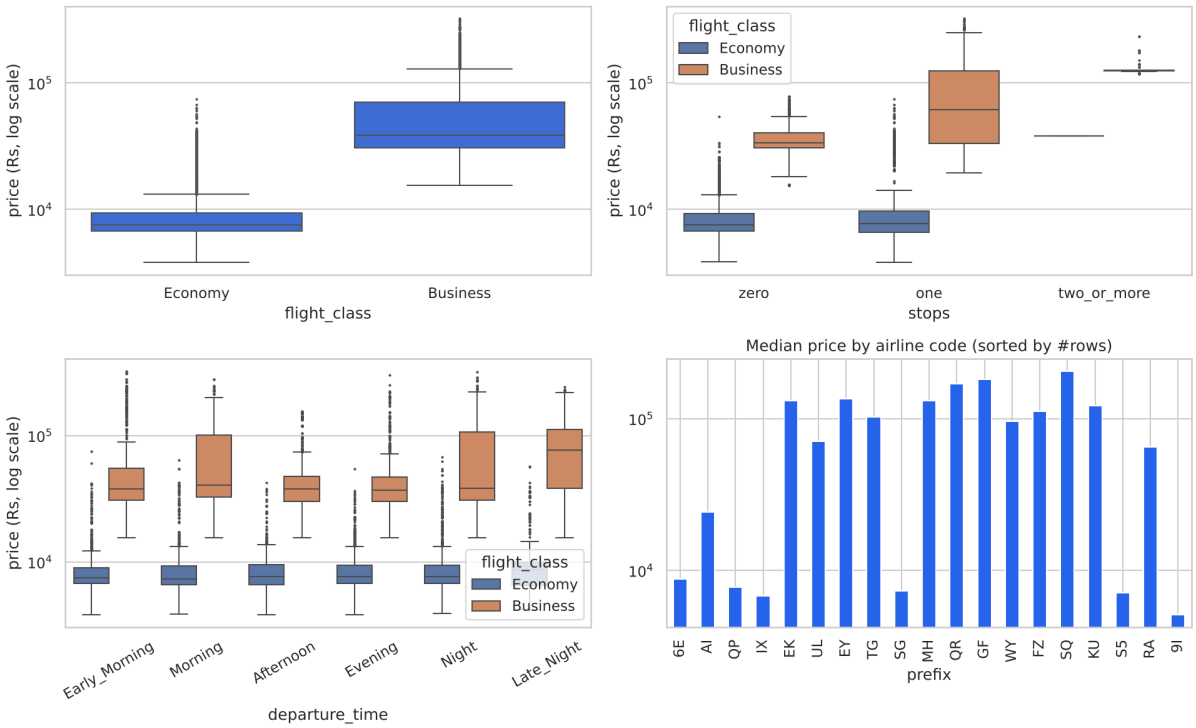


Figure 2: Price by class, stops, departure slot and airline code (log scale).

- **Airline code.** Seven codes (6E, AI, QP, IX, SG, S5, 9I) are low-fare carriers with medians below Rs 25,000; long-haul codes such as EK, EY, QR, SQ and GF have medians above Rs 100,000.

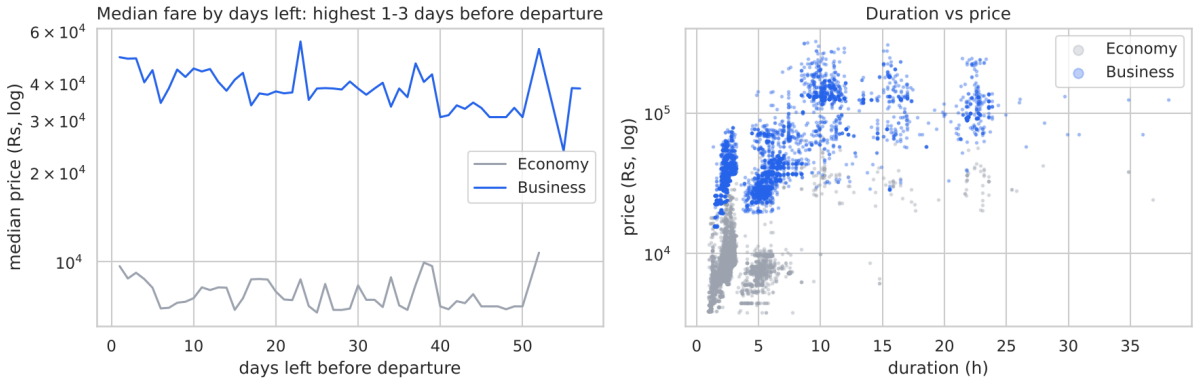


Figure 3: Median fare by days left (left) and duration vs price (right), by class.

- **Days left.** Fares are highest 1–3 days before departure: median Rs 48,562 (Business) and Rs 9,099 (Economy), 29% and 22% above the 31+ day medians. Beyond a week the curve is flat, which is why the linear correlation is only 0.01.
- **Duration.** Longer flights cost more (correlation with price 0.699); the clusters at 10, 15 and 22 h are long-haul one-stop Business itineraries.

Train vs test. The test file closely matches the training file: class share 59.9% / 40.1% vs 59.1% / 40.9%, the same `days_left` range (1–57), a duration range of 0.92–29.08 h (train 0.92–38.08 h), and about 20% missing airline and duration in both. Figure 5 overlays the distributions, and Figure 6 compares missingness.



Figure 4: Correlations with log-price — class 0.88, duration 0.64, stops 0.58, `days_left` 0.01, `id` 0.01.

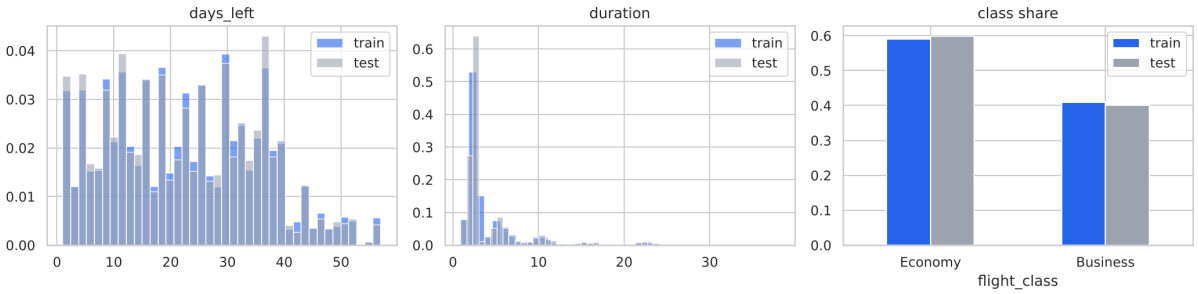


Figure 5: `days_left`, duration and class share — training set (blue) vs test set (grey).

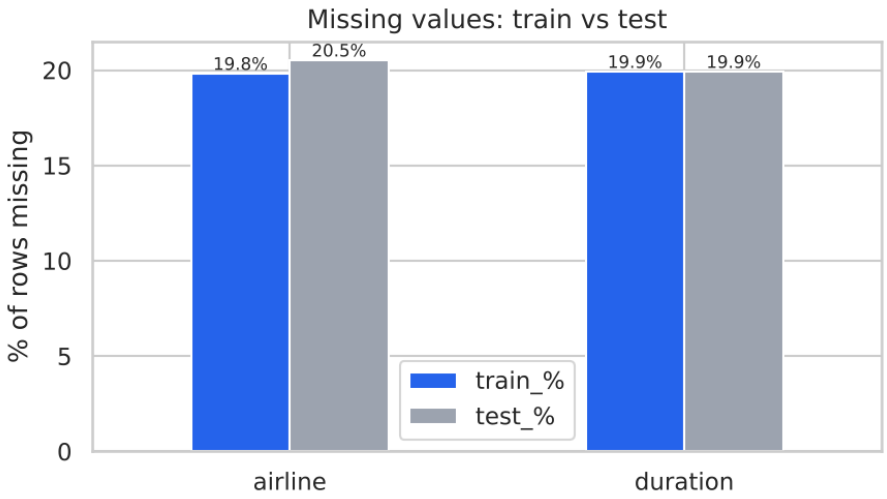


Figure 6: Share of missing values per column; only `airline` and `duration` have gaps.

5 Feature Engineering

The features that mattered were built during preprocessing: the log target, the airline code taken from the flight number, and the directional route. Seven further log, ratio and interaction features, tested afterwards, did not improve the blend.

5.1 Log-Transformed Features

- **Target.** `price` → `log1p(price)`. Skewness falls from 2.70 to 0.71 on the training data (the remaining skew comes from the two class humps), and squared error on the log scale penalises a 10% miss equally on a Rs 5,000 and a Rs 100,000 fare.
- **Inputs.** Tree models are invariant to monotone transforms, so log(duration) and log(days_left) were not used in the project models.

5.2 Ratio Features

None were used in the project models. Two were tested retrospectively: duration per stop (duration/(stops + 1)) and urgency (1/days_left).

5.3 Interaction Features

- **Route =** `source_city` × `destination_city` (30 one-hot columns). It replaced the 12 separate city columns because fares are direction-dependent: A→B and B→A are priced differently.
- **Airline code** extracted from `flight` (19 codes), which replaced the noisy `airline` column.
- Class interactions (class × `days_left`, class × duration, class × stops) were tested retrospectively.

Feature set	log RMSE (± sd)	MAE (Rs)	MAPE (%)
65 base features	**0.1352** (± 0.0054)	**3,341**	**7.93**
+ log(duration), log(days_left), duration per stop, 1/days_left, class × days_left, class × duration, class × stops	0.1364 (± 0.0048)	3,382	7.93

Table 4: Effect of seven extra features on the XGB + ET blend (5-fold CV, round-1 training split) *[retrospective]*.

The extra features made the blend slightly worse. Both members are tree ensembles that already find such splits and interactions on their own, so the extra columns add little information and slightly more noise.

6 Train–Validation Split

Each preprocessed training set was split 80/20 with

```
train_test_split(test_size=0.2, random_state=42, stratify=flight_class),
```

so both parts keep the 59/41 Economy/Business mix. Model selection used 5-fold `KFold(shuffle=True, random_state=42)` on the 80% part; the 20% hold-out was scored once per model as a second check.

Data	Rows after de-duplication	Training split	Hold-out split	Features
Round 1	14,431	11,544	2,887	65
Round 2	14,697	11,757	2,940	58

Table 5: Rows and features in the training and hold-out splits of each preprocessing round.

The Kaggle test file (3,898 rows) was never used for fitting. It received the same transforms, re-indexed to the training columns, and was predicted only for submissions. The final model was refitted on all 14,431 round-1 rows before predicting it.

Three caveats apply. The hold-out was drawn after de-duplication, so near-identical flights can appear on both sides. The two rounds have different hold-out rows, so their hold-out scores are not directly comparable. Finally, cross-validation scores log-price, while the Kaggle score appears to be R^2 on raw price, which weights expensive Business fares far more.

7 Supervised Models (Full Dataset)

Gradient-boosted and randomised tree ensembles all reach a cross-validated log RMSE of 0.134–0.146, about half the error of Ridge regression and a third of the class-mean baseline. XGBoost is the best single model on both data rounds, and averaging it with Extra Trees lowers the error further.

7.1 Models and settings

Model	Key settings
Baseline	Mean log-price of the flight's class (learned on the training split)
Ridge	`StandardScaler` + `Ridge(alpha=1.0)`
Random Forest	300 trees, `min_samples_leaf=2`
Hist Gradient Boosting	500 iterations, `learning_rate=0.1`
Hist Gradient Boosting (tuned)	`RandomizedSearchCV`, 20 settings × 5 folds → 800 iterations, lr 0.03, 63 leaves, `min_samples_leaf` 10, l2 0.1
Extra Trees	300 trees, `min_samples_leaf=2`
LightGBM	800 trees, lr 0.03, 63 leaves, `min_child_samples` 10, `reg_lambda` 0.1
XGBoost	800 trees, lr 0.03, `max_depth` 8, `subsample` 0.8, `colsample_bytree` 0.8
CatBoost	800 iterations, lr 0.05, depth 8
Voting (XGB + ET)	Unweighted mean of the XGBoost and Extra Trees predictions
Voting (XGB + ET + HGB)	Unweighted mean of three models

Table 6: Models compared in the project notebooks and their key hyper-parameter settings.

7.2 Round-1 data (`flight_ML`)

Model	log RMSE	R^2 (log)	MAE (Rs)	MAPE (%)
Baseline (class mean)	0.4921	0.7642	15,380	34.7
Ridge	0.2494	0.9394	6,552	17.3
Random Forest	0.1495	0.9782	3,784	8.4
Hist Gradient Boosting	0.1539	0.9769	4,166	9.7
Hist Gradient Boosting (tuned)	0.1498	0.9781	3,958	9.2
Extra Trees	0.1417	0.9804	**3,676**	**8.1**
LightGBM	0.1472	0.9789	3,944	8.9
XGBoost	**0.1371**	**0.9817**	3,750	8.3
CatBoost	0.1441	0.9798	3,952	9.1

Table 7: Round 1 — results on the 20% hold-out (2,887 rows).

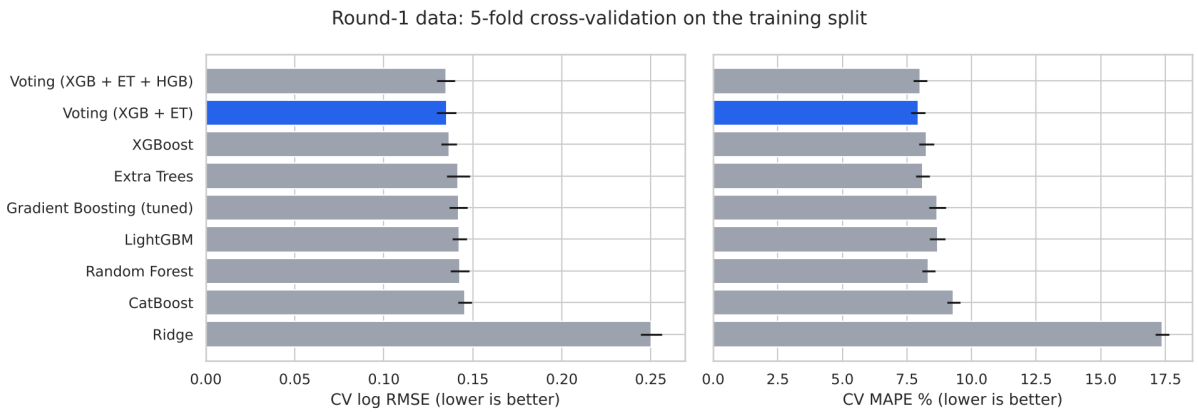


Figure 7: Round-1 5-fold CV log RMSE and MAPE with fold standard deviation (consolidated re-run); the submitted XGB + ET blend is highlighted.

Model	log RMSE (± sd)	log MAE	R^2 (log)	MAE (Rs)	MAPE (%)
Voting (XGB + ET + HGB)	**0.1349** (± 0.0051)	0.0810	**0.9818**	3,377	8.0
Voting (XGB + ET)	0.1352 (± 0.0053)	**0.0801**	0.9817	**3,339**	**7.9**
XGBoost	0.1364 (± 0.0043)	0.0829	0.9814	3,444	8.2
Extra Trees	0.1417 (± 0.0066)	0.0819	0.9799	3,410	8.1
HGB (tuned)	0.1418 (± 0.0051)	0.0872	0.9798	3,623	8.7
Random Forest	0.1425 (± 0.0053)	0.0834	0.9796	3,514	8.3
LightGBM	0.1425 (± 0.0041)	0.0873	0.9797	3,637	8.7
CatBoost	0.1456 (± 0.0039)	0.0941	0.9788	3,735	9.3
Ridge	0.2503 (± 0.0060)	0.1767	0.9372	6,379	17.4

Table 8: Round 1 — 5-fold cross-validation on the training split (11,544 rows), sorted by log RMSE.

7.3 Round-2 data (`flight_ML2`)

Model	log RMSE	R^2 (log)	MAE (Rs)	MAPE (%)
Baseline (class mean)	0.4703	0.7767	14,168	34.2
Ridge	0.2504	0.9367	6,291	17.4
Random Forest	0.1469	0.9782	3,584	8.1
Hist Gradient Boosting	0.1504	0.9772	3,863	9.1
Hist Gradient Boosting (tuned)	0.1495	0.9774	3,824	8.9
Extra Trees	0.1423	0.9796	**3,454**	**7.8**
LightGBM	0.1452	0.9787	3,675	8.5
XGBoost	**0.1394**	**0.9804**	3,497	8.1
CatBoost	0.1453	0.9787	3,726	9.0

Table 9: Round 2 — results on the 20% hold-out (2,940 rows).

7.4 Reading the results

- **Round 2 looked better offline but not on Kaggle.** Every tree model's CV log RMSE is 0.002–0.004 lower on round-2 data, yet every round-2 submission scored lower on the leaderboard (Section 10). The round-2 offline gain is no larger than the fold-to-fold spread (standard deviation 0.002–0.007), so it was not a reliable signal.
- **Extra Trees has the lowest rupee error, XGBoost the lowest log error.** The two make different mistakes, which is why averaging them helps.
- **The three-model blend is a statistical tie with the two-model blend** (0.1349 vs 0.1352, sd 0.005); the simpler XGB + ET blend was kept.

Tables 7–10 are copied from the project notebooks. The consolidated notebook re-runs every model with the same seeds and reproduces them closely: for round 1, every CV log RMSE matches to within 0.0002 and every hold-out log RMSE to within 0.0004 (for example XGBoost hold-out 0.1367 vs 0.1371). Its tables are saved in the notebook's outputs folder.

Model	log RMSE (± sd)	log MAE	R^2 (log)	MAE (Rs)	MAPE (%)
Voting (XGB + ET), OOF	**0.1329**	**0.0785**	**0.9824**	**3,319**	**7.8**
XGBoost	0.1336 (± 0.0039)	0.0815	0.9822	3,385	8.1
HGB (tuned)	0.1396 (± 0.0024)	0.0861	0.9805	3,611	8.6
Extra Trees	0.1399 (± 0.0050)	0.0801	0.9804	3,419	8.0
Random Forest	0.1405 (± 0.0039)	0.0821	0.9803	3,482	8.2
LightGBM	0.1405 (± 0.0030)	0.0864	0.9803	3,609	8.6
CatBoost	0.1417 (± 0.0036)	0.0916	0.9799	3,695	9.1
Ridge	0.2534 (± 0.0038)	0.1779	0.9359	6,585	17.6

Table 10: Round 2 — 5-fold cross-validation on the training split (11,757 rows).

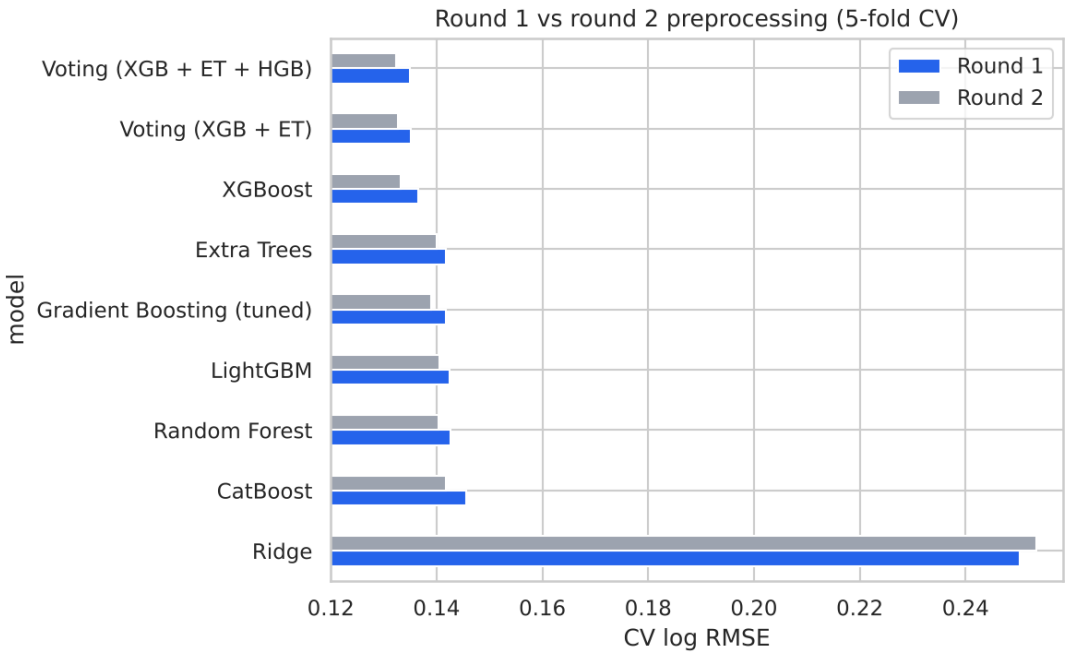


Figure 8: CV log RMSE of each model on round-1 (blue) and round-2 (grey) data (consolidated re-run).

8 Final Model Selection

The final model is `VotingRegressor([("xgb", XGBoost), ("et", ExtraTrees)])` trained on round-1 data with the Section 7.1 settings. It has the lowest rupee MAE and MAPE in cross-validation (Rs 3,339, 7.9%), ties the best log RMSE, and gave the best Kaggle score (0.96238).

8.1 Why this blend

- **Accuracy.** Among single models XGBoost has the lowest log RMSE (0.1364) and Extra Trees the lowest rupee MAE (Rs 3,410); the blend beats both on every metric.
- **Diversity.** Boosting (sequential, bias-reducing) and Extra Trees (randomised splits, variance-reducing) err in different places, so their average cancels part of each model's error.
- **Simplicity.** Adding tuned HGB as a third member changed log RMSE by only 0.0003, within one standard deviation.
- **Leaderboard confirmation.** The blend beat both of its members on Kaggle in both data rounds.

8.2 Hyper-parameter tuning (`flight_ml3`)

Tuning used `RandomizedSearchCV` with 5-fold CV on the round-1 training split, first scored on log RMSE and then on R^2 of the raw price, which appears to be the Kaggle metric.

Search	Model	Best setting found	CV score: tuned vs untuned	Outcome
Log RMSE, 30 settings	XGBoost	—	log RMSE 0.1374 vs 0.1364	Worse; kept untuned
Log RMSE, 20 settings	Extra Trees	300 trees, `min_samples_leaf` 3, all features, no bootstrap	log RMSE 0.1414 vs 0.1417	Negligible gain
Raw-price R^2, 30 settings	XGBoost	800 trees, lr 0.03, depth 6, colsample 0.5	R^2 0.9325; log RMSE 0.1440 vs 0.1364	Worse on log scale
Raw-price R^2, 20 settings	Extra Trees	300 trees, leaf 2, depth 30, bootstrap	R^2 0.9343; log RMSE 0.1414 vs 0.1417	Small gain

Table 11: Randomized-search tuning results for the two blend members, scored on log RMSE and on raw-price R^2.

The final `flight_ml3` run widened the search spaces and compared tuned models against *default* models (300 trees, library defaults) on the 20% hold-out:

Tuning helped Extra Trees on every metric. For XGBoost it raised raw-price R^2 but worsened log RMSE, MAE and MAPE: the R^2 scorer rewards fitting the few very expensive Business fares at the cost of everything else. Note that the default models here are weaker than the project's hand-set models (the project XGBoost has 800 trees and depth 8), so the gain over the submitted blend members is smaller than Table 12 suggests; Section 9 measures it directly.

Model	CV R^2 (Rs)	Hold-out R^2 (Rs)	log RMSE	MAE (Rs)	MAPE (%)
Tuned Extra Trees (700 trees, `max_depth` 30, `max_features` 0.85, `min_samples_split` 6, bootstrap)	**0.9344**	**0.9256**	**0.1442**	**3,711**	**8.45**
Tuned XGBoost (400 trees, lr 0.143, depth 4, subsample 0.70, colsample 0.59)	0.9303	0.9215	0.1514	4,074	9.84
Default XGBoost (300 trees)	—	0.9170	0.1451	3,901	8.65
Default Extra Trees (300 trees)	—	0.9041	0.1554	3,906	8.54

Table 12: Hold-out comparison after the widened search (raw-price R^2 scorer).

8.3 Error analysis of the final blend

Out-of-fold predictions on the training split (each row predicted by a model that never saw it) give a median absolute error of 4.2% and a mean of 7.9%; 75% of flights are within 10%. The errors concentrate in three places.

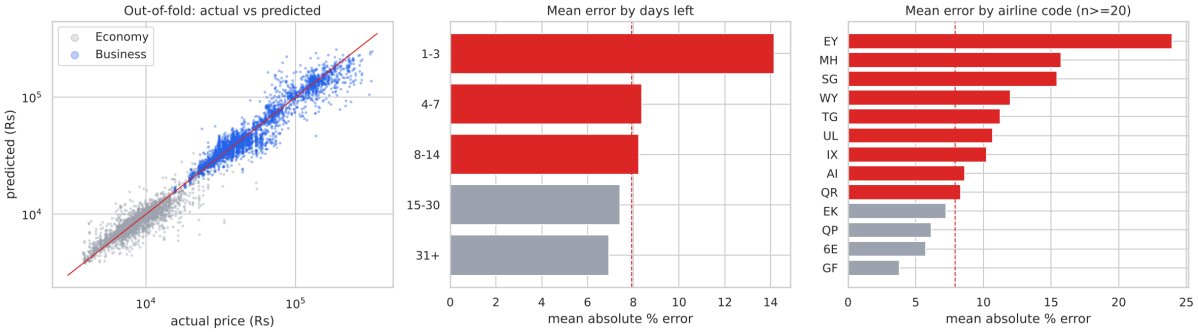


Figure 9: Out-of-fold actual vs predicted price (left) and mean absolute % error by days left and airline code (red = above the 7.9% average).

Group	Rows	Mean error (%)	Share of rows (%)	Share of total error (%)
Business class	4,764	9.8	41.3	50.9
Economy class	6,780	6.6	58.7	49.1
1–3 days left	716	14.2	6.2	11.1
31+ days left	3,420	6.9	29.6	25.9
Airline code EY	300	24.0	2.6	—

Table 13: Out-of-fold absolute percentage error of the final blend by group, with each group's share of rows and of total error.

- **Business fares are noisier, not harder to model.** Training rows with identical features differ in log-price by a standard deviation of 0.131 for Business and 0.058 for Economy, so part of the Business error is irreducible with these features.
- **Last-minute fares** (1–3 days) have twice the average error; the model shows no systematic over- or under-prediction there, only wider spread.
- **EY** flights are 91% Business, average 1.0 stop, 13.8 h and Rs 140,049. The `flight_ML` deep dive concluded that this error is mostly price noise among near-identical itineraries rather than a missing feature.
- **Separate models per class** did not help: CV log RMSE 0.1351 vs 0.1352 for the joint blend in `flight_ML` (0.1372 vs 0.1353 in the consolidated re-run), and Kaggle 0.96051 vs 0.96238.

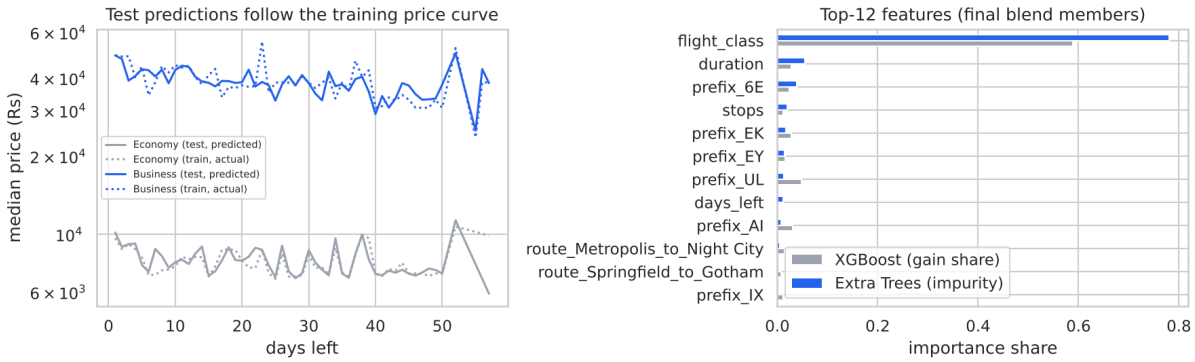


Figure 10: Left — median predicted test-set price by days left (solid) follows the training curve (dotted). Right — the 12 most important features of the two blend members.

Cabin class carries 78% of Extra Trees' impurity importance and 59% of XGBoost's gain, followed by duration, airline codes (6E, EK, UL, AI, EY) and stops. `days_left` ranks eighth in Extra Trees, consistent with its weak, last-week-only effect in Section 4.

9 Threshold Optimization

Price prediction is regression, so there is no decision threshold to tune as in classification. The project's two threshold-like choices were checked instead: the weight given to each blend member, and the 50-row cut-off for grouping rare airline codes. Neither offers a meaningful gain over the defaults used. *[retrospective]*

9.1 Blend weight

The XGBoost weight w (Extra Trees gets $1 - w$) was swept from 0 to 1 on out-of-fold predictions from the round-1 training split.

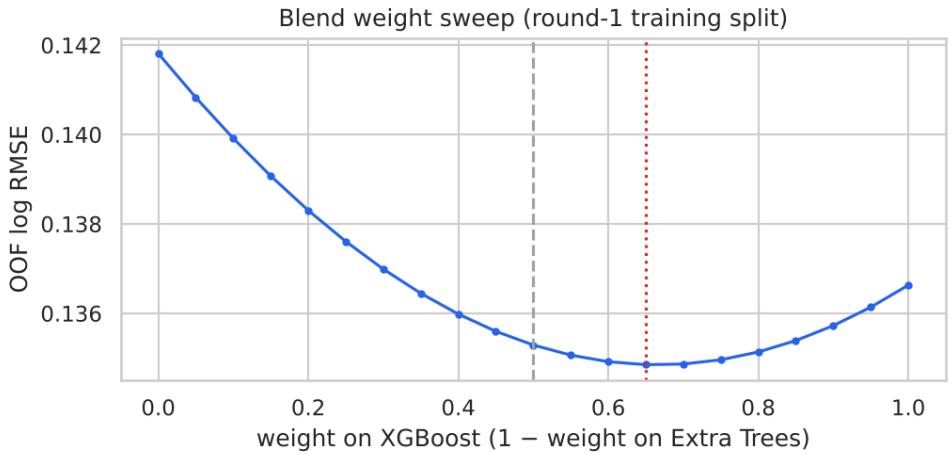


Figure 11: Out-of-fold log RMSE against the XGBoost weight; grey dashed = equal weights used in the submission, red dotted = minimum.

Weight on XGBoost	OOF log RMSE	OOF MAE (Rs)	OOF MAPE (%)	Hold-out log RMSE
0 (Extra Trees only)	0.1418	3,411	8.10	—
0.45	0.1356	**3,339**	**7.92**	—
0.50 (submitted)	0.1353	3,341	7.93	0.1354
0.65 (lowest log RMSE)	**0.1349**	3,357	7.98	**0.1349**
1 (XGBoost only)	0.1366	3,449	8.26	—

Table 14: Out-of-fold and hold-out errors for selected XGBoost weights in the blend.

The curve is flat between 0.45 and 0.75. A 0.65 weight lowers log RMSE by 0.0004 but raises rupee MAE and MAPE, so equal weights remain a sound choice.

9.2 Tuned members in the blend

Replacing the blend's Extra Trees with the tuned Extra Trees from Section 8.2 raised hold-out raw-price R^2 from 0.9259 to 0.9276 but worsened log RMSE (0.1354 → 0.1365), MAE (Rs 3,621 → 3,634) and MAPE (7.92% → 8.09%). The submitted blend already matches the best tuned single model on raw-price R^2 (0.9259 vs 0.9252) and beats it on every other metric.

9.3 Rare-code cut-off

The 50-row cut-off in round 2 affects 92 training rows and 29 test rows (0.7% of the test set). Round 2 as a whole scored below round 1 on Kaggle, so no further cut-offs were explored.

10 Kaggle Submission

Ten submissions were scored on the public leaderboard; the best, **0.96238**, is the round-1 XGBoost + Extra Trees blend. For every model submitted on both data rounds, round 1 scored higher, by 0.0015–0.0031.

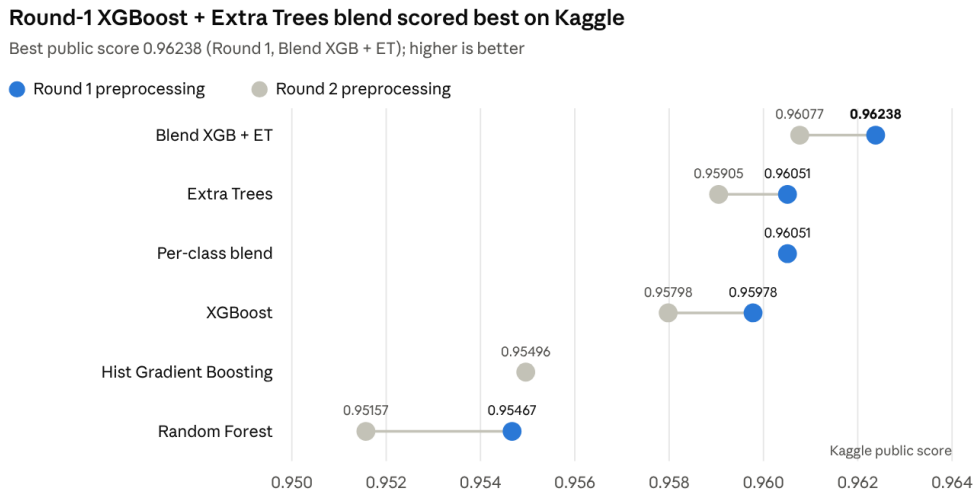


Figure 12: Kaggle public leaderboard scores recorded for each submission (`flight_ML`, `flight_ML2`).

Each submission file has two columns, `id` and `price` (rupees), with one row per test flight (3,898 rows). Predictions are made on the log scale and converted back with `expm1`. The blend submitted was fitted on the round-1 training data; the per-class blend (separate models for Economy and Business) improved cross-validated MAPE slightly but scored lower on Kaggle (0.96051), so the joint blend was kept.

10.1 Predictions on the test set

The consolidated notebook refits the blend on all 14,431 round-1 rows and writes `submission.csv` for the 3,898 test flights. The predicted prices range from Rs 3,850 to Rs 256,703 (median Rs 10,025, mean Rs 29,371), close to the training distribution (median Rs 10,183, mean Rs 29,817).

Class	Test rows	Training median (Rs)	Predicted test median (Rs)
Economy	2,337	7,587	7,740
Business	1,561	38,546	39,072

Table 15: Median training price and median predicted test-set price for each cabin class.

The predictions keep the two-hump shape and the class gap, and their median by days left tracks the training curve (Figure 10). The test labels are hidden, so the leaderboard score is the only accuracy measure on these rows.

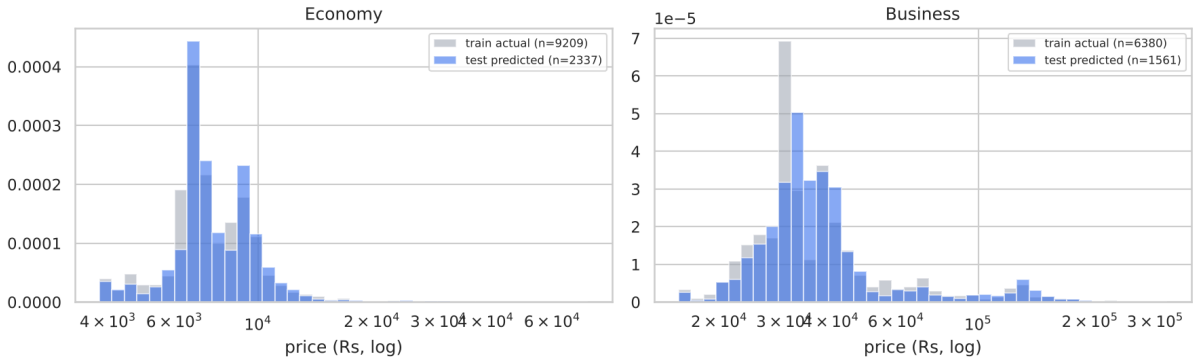


Figure 13: Distribution of predicted test-set prices (blue) against actual training prices (grey), by class.

11 Experiments with 20% Dataset

With only 20% of the training split (2,308 rows) the blend's hold-out log RMSE rises from 0.1354 to 0.1823 and MAPE from 7.9% to 10.3%, but it remains the best model and the ranking of models is unchanged. *[retrospective — the project notebooks always used the full data]*

Each model was trained on a stratified 20% sample of the round-1 training split and scored on the same 2,887-row hold-out as Table 7.

Model	log RMSE	R^2 (log)	MAE (Rs)	MAPE (%)
Blend XGB + ET	**0.1823**	**0.9676**	**4,580**	**10.3**
XGBoost	0.1860	0.9663	4,690	10.8
Extra Trees	0.1915	0.9643	4,721	10.6
Random Forest	0.1958	0.9627	4,864	11.1
Ridge	0.2531	0.9376	6,614	17.1

Table 16: Models trained on 20% of the training split, scored on the round-1 hold-out.

Ridge barely changes (0.2531 vs 0.2494 on full data): a linear model has few parameters and saturates early. The tree ensembles' log RMSE rises by 31–36%, so they are still data-limited.

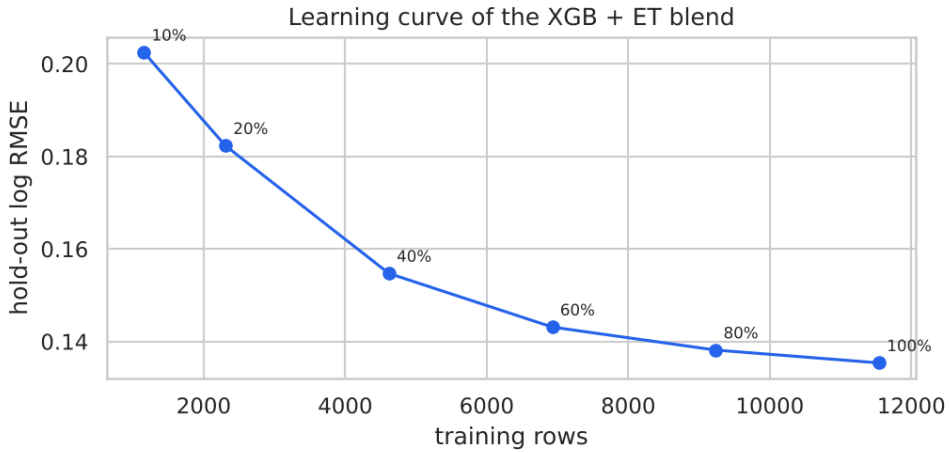


Figure 14: Hold-out log RMSE of the XGB + ET blend against the number of training rows (10%–100% of the training split).

The learning curve is still falling at 11,544 rows (0.1381 at 80% → 0.1354 at 100%), so more data would likely help further; Section 15 returns to this.

12 Unsupervised Learning

PCA and k-means on the 65 standardised round-1 features find one clear structure: a split between short, non-stop, mostly Economy flights and long, one-stop, mostly Business flights. Adding that cluster label as a feature did not improve the blend. *[retrospective]*

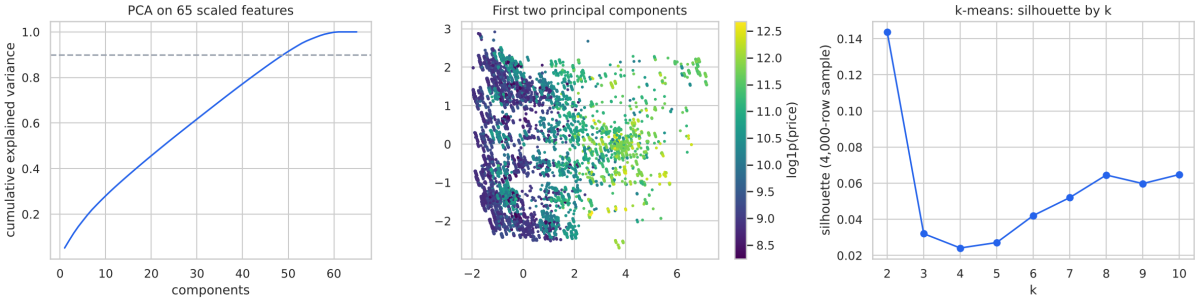


Figure 15: Cumulative PCA variance (left), the first two components coloured by log-price (centre), and k-means silhouette for $k = 2$–10 (right).

- **PCA.** Variance is spread thinly: 42 components are needed for 80% of the variance, 49 for 90% and 53 for 95%, because most features are sparse one-hot columns. Even so, the first component already orders flights from cheap (left) to expensive (right).
- **k-means.** The silhouette score peaks at $k = 2$ (0.144) and falls below 0.07 for every larger k, so only two natural groups exist.

Cluster	Rows	Median price (Rs)	Business share	Mean stops	Mean duration (h)	Mean days left
0 — short-haul	10,517	8,817	24%	0.05	2.5	22.3
1 — long-haul / connecting	3,914	46,420	87%	1.02	9.4	22.0

Table 17: The two k-means clusters (all 14,431 round-1 rows).

The clusters were found without looking at price, yet their median prices differ by 5×. Days left is the same in both, confirming it is unrelated to route structure. Added as a 66th feature, the cluster id left the blend's CV log RMSE unchanged within noise (0.1357 vs 0.1352), because the tree models already separate these groups through class, stops and duration.

13 Key Insights & Discussion

1. **Cabin class is most of the problem.** A class-mean baseline already reaches R^2 0.764 on log-price, and class carries 59–78% of the final models' importance. Everything else refines the price within a class.

2. **The log target was essential.** It turns an 85× price range into relative errors and lets one model serve Rs 4,000 and Rs 300,000 fares.

3. **Domain-driven cleaning beat clever imputation on the leaderboard.** Deriving airline from the flight prefix and building directional routes were the most useful steps. The ML-based duration fill of round 2 improved offline CV by 0.002–0.004 but scored lower on Kaggle every time.

4. **Diverse ensembles beat tuned single models.** Averaging XGBoost and Extra Trees helped more than any hyper-parameter search, and the blend beat both members in both rounds.

5. **The optimisation metric matters.** Tuning for raw-price R^2 favoured expensive Business fares and made XGBoost worse on log RMSE, MAE and MAPE.

6. **Remaining error is partly irreducible.** Identical Business itineraries differ in log-price by 0.131 on average, which caps achievable accuracy without new information such as booking time or seat availability.

14 Limitations

- **Offline/online mismatch.** Model selection used log RMSE, while the leaderboard appears to use raw-price R^2; round 2 won offline but lost online.
- **Single hold-out per round.** The two rounds were split separately, so their hold-out scores are not strictly comparable, and the hold-out was drawn after de-duplication, so near-identical flights can sit on both sides.
- **Leaderboard evidence is thin.** Differences between top submissions (≤ 0.002) are small and come from the public part of the test set only.
- **Fixed blend members.** The submitted blend used hand-set hyper-parameters; tuned members were only evaluated on the hold-out.
- **Missing context.** Dates, booking times and seat inventory are absent, which limits how well last-minute and Business fares can be predicted.
- **Retrospective analyses** (Sections 5, 9, 11, 12) were run after the submissions and did not influence the submitted model.

15 Future Work

- Optimise directly for the leaderboard metric, for example by training on raw price with a Business-weighted loss, or by checking each candidate with raw-price R^2 in cross-validation.
- Use repeated or grouped cross-validation (grouping duplicate itineraries) for more reliable comparisons between preprocessing rounds.
- Learn blend weights by stacking (a ridge meta-model on out-of-fold predictions) and include tuned members.
- Model the last-minute effect explicitly with finer `days_left` bands or monotonic constraints.
- Collect more data: the learning curve (Figure 14) is still falling at full size.

16 Conclusion

An equal-weight blend of XGBoost and Extra Trees, trained on log-price with the round-1 preprocessing, is our final model. It reaches a cross-validated log RMSE of 0.1352 (MAPE 7.9%, MAE Rs 3,339) and the team's best public Kaggle score of 0.96238. A second preprocessing round and two rounds of hyper-parameter tuning were tested carefully; neither beat this model on the leaderboard, although tuning did improve Extra Trees on its own. The consolidated notebook reproduces the round-1 results within 0.0005 log RMSE (round 2 within 0.003) and produces the final test-set predictions.

References

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] P. Geurts, D. Ernst and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of KDD 2016*, pp. 785–794, 2016.

[5] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017.

[6] L. Prokhorenkova et al., "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31*, 2018.

[7] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

Appendix A. Kaggle Public Leaderboard History

Notebook	Data	Model submitted	Public score
`flight_ML`	Round 1	**VotingRegressor(XGBoost, Extra Trees)**	**0.96238**
`flight_ML2`	Round 2	VotingRegressor(XGBoost, Extra Trees)	0.96077
`flight_ML`	Round 1	Extra Trees	0.96051
`flight_ML`	Round 1	Separate XGB + ET blends per class	0.96051
`flight_ML`	Round 1	XGBoost	0.95978
`flight_ML2`	Round 2	Extra Trees	0.95905
`flight_ML2`	Round 2	XGBoost	0.95798
`flight_ML2`	Round 2	Hist Gradient Boosting	0.95496
`flight_ML`	Round 1	Random Forest	0.95467
`flight_ML2`	Round 2	Random Forest	0.95157

Table 18: Public leaderboard score of every Kaggle submission, listed by notebook, data round and model.

No Kaggle score is recorded for the tuned models from `flight_ml3`; they were compared on the hold-out split only.

Appendix B. Artifact Traceability

Code repository

The complete project codebase, notebooks, submission files and this report are stored in the following GitHub repository:

https://github.com/halconie/ml-project-1

Project artifacts

Artifact	Produced by	Used in
`flight_preprocessing.ipynb` → `1stRoundFlightdata.csv` (14,431 × 66)	Round-1 preprocessing	Sections 3, 7–10
`flight_preprocessing2.ipynb` → `2stRoundFlightdata.csv` (14,697 × 59)	Round-2 preprocessing	Sections 3, 7
`flight_ML.ipynb`	Round-1 model comparison, blends, error analysis	Sections 7, 8, 10
`flight_ML2.ipynb`	Round-2 model comparison	Sections 7, 10
`flight_ml3.ipynb` → `xgboost_tuning_results.csv`, `extra_trees_tuning_results.csv`	Randomized-search tuning	Section 8
`Flight_Price_Prediction_Final.ipynb`	Consolidated re-run of all five notebooks + retrospective analyses	All figures; Sections 5, 9, 11, 12
`figures/*.png`	Consolidated notebook	Figures 1–15
`outputs/*.csv`	Consolidated notebook (metric tables)	Tables in Sections 3–12
`submission.csv` (3,898 rows: `id`, `price`)	Final blend refitted on all round-1 rows	Section 10